SOC Lab Conteneurisé

Déploiement et exploitation d'un centre opérationnel de sécurité

Wazuh 4.14.3 | Suricata | TheHive 5 | Grafana | Docker Compose

Champ	Information
Nom & Prénom	Mouhamed Moustapha Sabaly & Penda Sall Pouye
Module	Cloud et Infrastructures Virtuelles
Formation	L2 SIMAC P11
Date de rendu	19 Mars 2026
Dépôt Git	soc-lab-Sabaly-SallPouye/
Environnement	Ubuntu 24.04 LTS — VMware — 7.7 Go RAM
Stack	Wazuh 4.14.3, Suricata, TheHive 5.4, Grafana 10.2, DVWA

Résumé : Ce rapport documente le déploiement complet d'un SOC Lab conteneurisé sur Docker Compose, incluant Wazuh SIEM, Suricata IDS, TheHive 5 pour la gestion d'incidents, Grafana pour la visualisation, et DVWA comme cible d'attaque. Trois scénarios d'attaque ont été simulés : reconnaissance Nmap (TA0043), brute force SSH (T1110.001), et injection SQL (T1190).

1 Introduction

Contexte

Dans un contexte où les cybermenaces évoluent constamment, les entreprises doivent disposer d'un Security Operations Center (SOC) capable de détecter, analyser et répondre aux incidents en temps réel. Ce projet académique, réalisé dans le cadre du module Cloud et Infrastructures Virtuelles de L2 SIMAC, vise à concevoir et déployer un SOC Lab entièrement conteneurisé avec Docker Compose.

Presentation de TechCorp SA

TechCorp SA est une entreprise fictive de 350 employés opérant dans le secteur fintech. Elle héberge des données sensibles (informations clients, transactions bancaires) et est soumise aux exigences réglementaires PCI-DSS et RGPD. L'absence de supervision de sécurité l'expose à des risques de compromission non détectée.

Objectifs du projet

- Déployer une stack de sécurité complète en une seule commande Docker Compose
- Configurer la segmentation réseau selon le principe du moindre privilège
- Simuler 3 scenarios d'attaque realistes (Nmap, Hydra SSH, SQLi)
- Détecter et corréler les alertes dans Wazuh SIEM et Suricata IDS
- Gérer le cycle de vie des incidents dans TheHive 5
- Produire un rapport technique documentant toutes les étapes

Perimetre technique

Service	Version	Role	Port
Wazuh Indexer	4.14.3	Stockage alerts (OpenSearch)	9200
Wazuh Manager	4.14.3	SIEM principal + Filebeat	55000, 1514
Wazuh Dashboard	4.14.3	Interface web SIEM	443
Suricata	latest	IDS/IPS reseau	host network
TheHive	5.4	Gestion incidents	9000
Grafana	10.2.0	Visualisation métriques	3000
Cassandra	4.1	Base de donnees TheHive	9042
DVWA	1.10	Cible vulnerable	8080

2 Architecture Docker

Schema d'architecture

```
+-------------------------------------------------------------+
|         HOTE UBUNTU 24.04 -- 192.168.1.40                   |
|         vm.max_map_count=262144  |  Docker 28.2             |
+-------------------------------------------------------------+
| soc-frontend (172.19.0.0/16)                                |
| [Wazuh Dashboard:443] [Grafana:3000] [TheHive:9000] [DVWA:8080]|
+-------------------------------------------------------------+
| soc-backend (172.18.0.0/16)                                 |
| [Wazuh Indexer:9200] [Wazuh Manager:55000] [Cassandra:9042]  |
| [Suricata IDS -- host network -- ens33]                     |
+-------------------------------------------------------------+
| Volumes: wazuh-indexer-data  cassandra-data  grafana-data   |
|          thehive-data  thehive-index  suricata-logs         |
+-------------------------------------------------------------+
```

Figure 1 -- Architecture SOC Lab conteneurise avec segmentation reseau Docker

Justification des choix architecturaux

Segmentation réseau: soc-frontend (services utilisateurs) et soc-backend (composants sensibles). Le Dashboard est sur les deux réseaux pour accéder à l'Indexer. Aucun service interne n'est exposé directement.

Suricata en mode host network: network_mode: host permet à Suricata d'accéder à l'interface physique en 33. En mode bridge, il ne verrait que le trafic Docker interne.

vm.max_map_count=262144: OpenSearch utilise mmap pour ses index. La valeur par défaut Linux (65530) est insuffisante. Rendu persistant via /etc/sysctl.conf.

Matrice de connectivité réseau

Source	Destination	Resultat	Justification
Grafana	wazuh.manager	**OK Joignable**	soc-backend commun
suricata	Grafana	**BLOQUE Injoignable**	host network isole
TheHive	wazuh.manager	**OK Joignable**	soc-backend commun
wazuh.dashboard	suricata	**BLOQUE Injoignable**	suricata = host net
wazuh.dashboard	wazuh.indexer	**OK Joignable**	double reseau dashboard
DVWA	Cassandra	**BLOQUE Injoignable**	reseaux differents

3 Déploiement -- Difficultés et Solutions

Le déploiement a nécessité de résoudre 6 problèmes techniques. Chaque difficulté est documentée avec la cause racine et la solution appliquée.

C1 -- YAML invalide ligne 32 -- healthcheck format compact

Cause : Le healthcheck de wazuh.indexer utilisait le format compact sur une ligne, invalide pour Docker Compose v2.

Solution : Separation en 4 lignes distinctes : interval: 30s | timeout: 10s | retries: 10 | start_period: 120s

```
docker compose config && echo 'YAML valide'
```

C2 -- Cassandra exited(1) -- GC incompatible

Cause : MAX_HEAP_SIZE + HEAP_NEWSIZE activent le GC CMS, incompatible avec G1GC de Java 11+ utilisé par Cassandra 4.x.

Solution : Suppression de ces variables + ajout volume cassandra-data:/var/lib/cassandra.

```
docker compose logs cassandra 2>&1 | tail -5
```

C3 -- Dashboard ENOENT -- opensearch_dashboards.yml manquant

Cause : Le container wazuh.dashboard cherche ses fichiers a des chemins précis non montés initialement.

Solution : Ajout de 5 volumes obligatoires : opensearch_dashboards.yml, wazuh.yml, et 3 certificats TLS.

```
docker compose logs wazuh.dashboard 2>&1 | grep ENOENT
```

C4 -- ERROR3099 -- Invalid credentials wazuh-wui

Cause : Le hash scrypt dans rbac.db ne correspond pas au mot de passe. Paramètres de hash différents.

Solution : Regeneration via werkzeug.security.generate_password_hash() + mise a jour sqlite3 dans rbac.db.

```
docker exec wazuh.manager python3 -c "from werkzeug.security import
generate_password_hash; ..."
```

C5 -- Filebeat -- username/password commentes

Cause : Le script d'init du container commente les credentials (#username:, #password:). Aucune alerte indexée.

Solution : sed -i pour décommenter dans filebeat.yml -- SANS pkill (pkill tuerait tout le container).

```
docker exec wazuh.manager bash -c "sed -i 's|  #username:|  username: admin|'
/etc/filebeat/filebeat.yml"
docker compose restart wazuh.manager
```

C6 -- TLS -- unable to verify the first certificate

Cause : Après régénération des certificats, l'indexer garde les anciens en mémoire. Chaîne de confiance rompue.

Solution : Recreation forcee : docker compose rm -sf wazuh.indexer && docker compose up -d wazuh.indexer.

```
docker compose rm -sf wazuh.indexer
docker compose up -d wazuh.indexer
```

4 Résultats des Attaques -- Timeline et Analyse

Timeline des 3 phases d'attaque -- 19/03/2026

Heure (GMT)	Phase	Outil	MITRE	Duree	Resultat
21:06:00	Reconnaissance	Nmap 7.94	TA0043	110s	7 ports decouverts
21:18:04	Brute Force SSH	Hydra v9.5	T1110.001	10s	14 alertes Wazuh
21:34:00	SQL Injection	Manuel Firefox	T1190	5 min	5 hachages MD5
22:03:00	Incident #1 TheHive	TheHive 5	--	--	Cas SSH [HIGH]
22:11:00	Incident #2 TheHive	TheHive 5	--	--	Cas Nmap [MEDIUM]
22:28:00	Incident #3 TheHive	TheHive 5	--	--	Cas SQLi [HIGH]

4.1 Phase 1 -- Reconnaissance Nmap (TA0043)

La reconnaissance est la première étape du Lockheed Martin Kill Chain. Commande : nmap -sV -sC -A localhost. Durée : 110 secondes.

Port	Service	Version	Observation
22/tcp	SSH	OpenSSH 9.6p1	Authentification par mot de passe -- cible brute force
443/tcp	HTTPS	Wazuh Dashboard	TLS valide, cert wazuh.dashboard (2036)
631/tcp	IPP	CUPS 2.4.7	Interface admin imprimante -- exposition inutile
3000/tcp	HTTP	Grafana	Redirect /login -- pas d'auth bypass evidente
8080/tcp	HTTP	Apache 2.4.25	DVWA v1.10 -- cookie PHPSESSID sans httponly [ALERTE]
9000/tcp	HTTP	TheHive API	REST API exposee -- pas de TLS
9200/tcp	HTTPS	OpenSearch	Basic Auth requis -- cert wazuh.indexer

4.2 Phase 2 -- Brute Force SSH Hydra (T1110.001)

Parametre	Valeur
Cible	ssh://localhost:22
Utilisateur cible	root
Wordlist	9 mots de passe (password, 123456, root, admin...)
Duree totale	10 secondes (21:18:04 --> 21:18:14)
Mots de passe valides	0 -- système correctement sécurisé

Alertes Wazuh generees	**14 (Authentication failure, rules 5710/5716/5763)**
Index OpenSearch cree	wazuh-alerts-4.x-2026.03.19 (224 documents)
Techniques MITRE detectees	T1110 Brute Force, T1110.001 Password Guessing, SSH

Wazuh Threat Hunting : Total 14 alertes | Authentication failure : 14 | Authentication success : 0 | MITRE : Password Guessing + SSH + Brute Force

4.3 Phase 3 -- Injection SQL DVWA (T1190)

DVWA configure en niveau Low Security. Deux payloads executes manuellement dans /vulnerabilities/sqli/ :

Payload	Type	Resultat
1' OR '1'='1	Tautologie	5 utilisateurs exfiltrés
%' AND 1=0 UNION SELECT null, concat(user,0x3a,password) FROM users#	UNION SELECT	5 hachages MD5 exfiltrés

Utilisateur	Hachage MD5	Mot de passe	Methode
admin	5f4dcc3b5aa765d61d8327deb882cf99	**password**	Rainbow table
gordonb	e99a18c428cb38d5f260853678922e03	**abc123**	Rainbow table
1337	8d3533d75ae2c3966d7e0d4fcc69216b	**charley**	Rainbow table
pablo	0d107d09f5bbe40cade3de5c71e9e9b7	**letmein**	Rainbow table
smithy	5f4dcc3b5aa765d61d8327deb882cf99	**password**	Rainbow table

4.4 Gestion des incidents -- TheHive 5

Cas #	Titre	Severite	Tags
#1	SSH Brute Force Attack -- 19/03/2026	**HIGH**	brute-force, ssh, t1110
#2	Nmap Reconnaissance Scan -- 19/03/2026	**MEDIUM**	nmap, reconnaissance, ta0043
#3	SQL Injection DVWA -- 19/03/2026	**HIGH**	sql-injection, dvwa, t1190

5 Analyse de Sécurité

Points forts de l'infrastructure

- **Segmentation réseau:** soc-frontend et soc-backend isolent les composants sensibles.
- **Secrets externalisés:** Tous les mots de passe sont dans .env, absent du dépôt Git.
- **TLS sur tous les endpoints:** Wazuh Dashboard et OpenSearch utilisent des certificats TLS.
- **Volumes nommés persistants:** Les données Waouh, Cassandra et Grafana survivent aux redémarrages.
- **Healthchecks:** Services critiques avec healthchecks evitant les demarrages prematures.
- **Détection MITRE ATTACK:** Wazuh mappe automatiquement les alertes aux techniques MITRE.

Points faibles identifiés

- **Certificats auto-signés:** Non signés par une CA reconnue. Let's Encrypt nécessaire en production.
- **Pas de réponse active Wazuh:** 14 alertes générées mais aucun blocage iptables automatique.
- **TheHive sans MFA:** Accessible sans authentification multi-facteurs. Mot de passe faible.
- **DVWA en Low Security:** Ne doit jamais être exposé sur un réseau non isolé.
- **Suricata sans regles custom:** fast.log vide. Regles Emerging Threats non chargees.

Suricata -- Analyse du trafic capture (eve.json)

Type d'evenement	Volume	Analyse
flow	17 920	Sessions TCP/UDP -- trafic Docker + hote
mdns	6 048	Multicast DNS -- services locaux Ubuntu (Avahi)
stats	2 320	Métriques internes Suricata
dns	1 610	Requetes DNS -- resolutions Docker + reseau hote
tls	220	Handshakes TLS -- communications Wazuh chiffrees
http	109	Requetes HTTP -- DVWA, Grafana, TheHive
fileinfo	46	Metadonnees fichiers transferes
quic	20	Protocole QUIC -- trafic Google/navigateur
dhcp	4	Requetes DHCP -- VMware sur ens33

Total : 16 913 lignes dans eve.json. fast.log vide car règles ET Open non chargées.

6 Recommandations de Sécurité

R1 -- Gestion des secrets -- HashiCorp Vault / Docker Secrets

Problème : Les credentials sont en clair dans .env sur le système de fichiers hôte. Compromission = exposition totale.

Solution : Déployer HashiCorp Vault en container. Les services récupèrent leurs secrets via l'API. Alternative : Docker Secrets (swarm mode) -- montes en tmpfs, jamais sur disque.

```
# Docker Secrets (swarm mode)
echo 'SecretPassword' | docker secret create opensearch_password -
# Dans docker-compose.yml
secrets:
  opensearch_password:
    external: true
```

R2 -- Durcissement des conteneurs -- Least Privilege

Problème : Les containers tournent avec des capacités Linux étendues. Wazuh Manager tourne en root.

Solution : Appliquer : read_only: true avec tmpfs pour /tmp, profil seccomp restrictif, user non-root (472:472), cap_drop: ALL.

```
# docker-compose.yml -- durcissement grafana
grafana:
  read_only: true
  user: '472:472'
  security_opt:
    - no-new-privileges:true
  cap_drop: [ALL]
  tmpfs: ['/tmp']
```

R3 -- Supervision -- Notifications Slack/email + Reponse active

Problème : Les alertes Wazuh restent dans le dashboard sans notification externe. Réponse active non configurée.

Solution : Configurer Wazuh pour envoyer des webhooks Slack sur alertes niveau >= 12. Activer la réponse active pour bloquer les IP apres 5 tentatives de brute force (iptables, timeout: 600s).

```
# wazuh/config/ossec.conf -- reponse active
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>5763</rules_id>
  <timeout>600</timeout>
</active-response>
```

7 Conclusion

Bilan des compétences acquises

- Orchestration multi-services avec Docker Compose (8 containers, 2 réseaux, 6 volumes)
- Diagnostic et resolution d'erreurs complexes (TLS, YAML, permissions, databases)
- Configuration d'un SIEM Wazuh complet avec indexer, manager, dashboard et Filebeat
- Simulation d'attaques réelles et analyse des alertes SIEM (MITRE ATTaCK mapping)
- Gestion d'incidents selon le workflow SOC dans TheHive 5 (cas, observables)
- Analyse de logs Suricata IDS (eve.json -- 16 913 événements réseau captures)
- Rédaction d'un rapport technique structure documentant toutes les étapes

Résultats quantitatifs

Metrique	Valeur
Services Docker deployes	8/8 -- tous healthy
Certificats TLS generes	11 (root-ca, admin, indexer, manager, dashboard)
Problemes techniques resolus	6 erreurs critiques (C1 -- C6)
Ports decouverts par Nmap	7 (22, 443, 631, 3000, 8080, 9000, 9200)
Alertes SSH brute force	14 alertes Wazuh -- MITRE T1110.001 detecte
Documents OpenSearch indexes	224 (wazuh-alerts-4.x-2026.03.19)
Hachages MD5 exfiltrés	5 comptes -- tous crackés via rainbow tables
Evénements Suricata captures	16 913 (flows, DNS, TLS, HTTP)
Incidents TheHive crees	3 cas (2 HIGH, 1 MEDIUM)

Extensions envisageables

- **MISP -- Threat Intelligence:** Intégration de MISP pour enrichir les alertes Wazuh avec les IoCs connus (+2 pts bonus).
- **Déploiement cloud (AWS/OVH):** Portage sur VM cloud pour tester la résilience réseau en production (+1 pt bonus).
- **Notifications Slack/email:** Integration Wazuh --> Slack Webhook pour alertes temps reel (niveau >= 12) (+1 pt bonus).
- **Regles Suricata custom:** Regles ET Open pour detecter SQLi, scans Nmap et brute forces SSH.

> Ce projet démontre qu'un SOC opérationnel peut être déployé rapidement grâce à la conteneurisation Docker. Les outils open-source (Wazuh, Suricata, TheHive) offrent des capacités comparables aux solutions commerciales SIEM.

A Annexes

A.1 Structure du depot Git

```
soc-lab-Sabaly-SallPouye/
|-- README.md                    <- Instructions de deploiement
|-- docker-compose.yml           <- Stack SOC complete (8 services)
|-- .env.example                 <- Variables fictives -- JAMAIS le .env
|-- .gitignore                   <- .env et certificats ignores
|-- deploy_soc_lab_partie1.sh    <- Script deploiement automatise
|-- deploy_soc_lab_partie2.sh    <- Script attaques automatisees
|-- wazuh/config/
|   |-- wazuh_cluster/wazuh_manager.conf
|   |-- wazuh_dashboard/opensearch_dashboards.yml + wazuh.yml
|   |-- wazuh_indexer/wazuh.indexer.yml + internal_users.yml
|   +-- wazuh_indexer_ssl_certs/  <- 11 certificats .pem
|-- suricata/rules/ + logs/      <- eve.json (16913 lignes), fast.log
|-- grafana/dashboards/
|-- thehive/config/
+-- rapport/
    |-- SOC LAB CONTENEURISÉ Guide complet de déploiement et simulation d'attaques.pdf
    |-- rapport-final-SOC-Lab-Sabaly-SallPouye.pdf
    +-- captures/                <- nmap_scan.txt, hydra_ssh.txt,
                                    sqlmap_scan.txt, resume_attaques.txt
```

A.2 Variables d'environnement (.env.example)

```
WAZUH_VERSION=4.14.3
INDEXER_USERNAME=admin
INDEXER_PASSWORD=CHANGE_ME_SecretPassword
WAZUH_API_USERNAME=wazuh-wui
WAZUH_API_PASSWORD=CHANGE_ME_MyS3cr37P450r
DASHBOARD_USERNAME=kibanaserver
DASHBOARD_PASSWORD=CHANGE_ME_kibanaserver
GRAFANA_ADMIN_USER=admin
GRAFANA_ADMIN_PASSWORD=CHANGE_ME_GrafanaAdmin
THEHIVE_SECRET=CHANGE_ME_32CharsSecretKeyMinimum
CASSANDRA_CLUSTER_NAME=thehive
```

A.3 Commandes essentielles SOC Lab

Commande	Description
docker compose up -d	Démarrer toute la stack en arrière-plan
docker compose ps	Vérifier l'état de tous les services
docker compose logs -f wazuh.manager	Suivre les logs du manager
docker compose restart wazuh.manager	Redémarrer le manager (apres fix)
docker compose rm -sf wazuh.indexer	Recreer l'indexer (fix TLS)
nmap -sV -sC -A localhost	Scan complet de l'hote
hydra -l root -P pass.txt ssh://localhost	Brute force SSH avec Hydra